BETAFLIGHT · STM32F722 · BITBANG BIDIR DSHOT600 · UPDATED 15 OCT 2026

Bringing back 8 kHz with bidir DShot600 on F722

PR #15779 forces DShot300 and a 4 kHz PID loop on F7 when bidirectional DShot is on. This page sums up what we measured, what we changed, and the "don't spin" fix that targets the real cause.

> **What "the spin" means.** With bidirectional DShot, each motor pin does two jobs every loop. It sends the motor frame (28 µs), then listens for the ESC's RPM reply (62 µs). If the next motor update arrives while the pin is still listening, today's code makes the CPU wait in a busy loop ("spin") until the listen window closes. At 8 kHz a loop is 125 µs, so one late loop can lose up to 62 µs to this wait and miss the next gyro sample. The "spin fixes" skip that one motor update instead of waiting: the ESC keeps its last throttle for one more loop, and the CPU goes on at once. The diagram under "The don't-spin fix" shows it.

Status: built and checked by AI agents. Not yet reviewed by a person. Never flown. Bench use only, props off.

Grades: MEASURED ran it on the board or in the simulator · READ from source with file:line · HYPOTHESIS everything else.

Bottom line

PID LOOP WORK, ALL PATCHES

−12.0%

8,033 → 7,071 cycles per loop. 5 interleaved board rounds. MEASURED

AVERAGE CPU AT 8 KHZ

49 → 45%

61.1 → 56.4 µs of a 125 µs loop. Measured at 3.2 kHz, projected to 8 kHz. HYPOTHESIS

WORST-CASE LOOP, DON'T-SPIN FIX

~123 → ~56µs

Today a late loop can spin up to 62 µs. The fix removes that wait. Needs an 8 kHz board to prove. HYPOTHESIS

The patches give back about 5 µs per loop. That helps, but the thing that decides whether 8 kHz is safe is the telemetry wait. The order of value is: the don't-spin fix first, then the cycle savings, then the scheduler headroom.

What one loop does

1. **Sense.** The gyro raises an interrupt when a sample is ready; DMA copies it over SPI with no CPU.
2. **Clean.** Low-pass filters, a dynamic notch, and the RPM filter remove motor and prop noise. The RPM filter needs each motor's RPM, which comes from bidir DShot telemetry.
3. **Decide.** The PID controller compares the stick request with the measured rotation.
4. **Mix.** The mixer turns roll, pitch, yaw and throttle into four motor commands.
5. **Act.** Bitbang DShot sends each ESC a 16-bit frame. With bidir on, the same wire then listens for the ESC's RPM reply.

At 8 kHz all five steps must fit in 125 µs, every time. The board's other jobs (RC input, OSD, blackbox, battery) fit in the gaps.

Board results: all patches together

`SpeedyBee F7 V3 (BMI270, 3.2 kHz) · USB only, no ESC · DShot600, bidir on, pid_process_denom 1 · cycle counter averaged over 1,024 calls · 5 interleaved rounds · base = master + test guard (68449a311), patched = f722-bidir-perf (A2-E)`

STAGE	BASE CYCLES	PATCHED	CHANGE	TIME
Motor update (mixer + DShot write + decode)	5,793	4,868	−16.0%	26.8 → 22.5 µs
Telemetry decode	1,714	1,399	−18.4%	7.9 → 6.5 µs
Gyro filtering	4,721	4,667	−1.2%	21.9 → 21.6 µs
PID controller	1,566	1,550	−1.0%	7.3 → 7.2 µs
RC command	399	385	−3.5%	1.9 → 1.8 µs
Gyro update	453	447	−1.3%	2.1 → 2.1 µs

Whole PID loop task	**8,033**	**7,071**	**−12.0%**	**37.2 → 32.7 µs**

Decode is counted inside motor update. Gyro filtering is a separate task, so it is not inside the PID loop total. Changes of about 1% are inside boot-to-boot noise.
MEASURED

Scheduler pass cost (patch A2)

Same board and settings · probe counts cycles per scheduler pass and passes per 100 ms · 3 rounds × 1,500 samples · base spread 1 cycle, ~14 passes

BUILD	CYCLES PER PASS	VS BASE	PASSES / 100 MS
base	632	—	18,124
A2 (time-driven task age)	**611**	**−3.3%**	**19,806 (+9.3%)**
Sol 1 (alternative scheduler change)	630	−0.3%	19,269 (+6.3%)
B / Sol 2 (hold-task checks)	632	0	18,116

A2 is repeatable: each pass is 21 cycles cheaper, so the scheduler checks for gyro data 9% more often. The earlier write-up said patch A could not be measured; this probe now measures it. B and Sol 2 show zero because the altitude-hold and position-hold tasks they touch are not enabled on this board. MEASURED

Patch by patch

PATCH	WHAT IT DOES	EVIDENCE	VERDICT
A2	Scheduler computes task age from time with one divide, with the same priority result as master (checked over 1.3 billion cases)	−3.3% per pass, +9% passes	Keep
B	Cheaper checks in the hold tasks	No effect here: tasks not running. Needs a GPS/baro setup	Untested here

C	Mixer: skips thrust-linear math when it is set to 0, hoists a failsafe check	No gain on F722 (the compiler already hoists it). Adds a branch when thrust linear is on	Drop
D	Faster bidir telemetry decoder	Simulator: −34% decoder instructions. Board, 3 rounds: decode −10.5%, loop −2.7%	Keep
E	Swaps colder code out of ITCM for hotter code (ITCM 15,712 B of 16 KB)	Board, 3 rounds: loop −8.2%, motor update −10.9%. The largest single gain	Keep
Sol 3	Replaces the do-nothing filter call for a disabled notch with a NULL check	Board, defaults: loop −2.1%. Gain only when notches are off; same math when on	Best case only
Sol 5	Moves the DShot DMA handler and telemetry update into ITCM (ITCM +416 B)	Built, not benched	Candidate

Upstream bug found on the way: `FAST_CODE_NOINLINE` is not placed in ITCM on STM32 (it expands to `NOINLINE` only), so `updateDshotTelemetry` runs from flash with 7 wait states. READ + map file MEASURED

Each patch alone on the board

Same board and settings · each build = master + guard + one patch · 3 interleaved rounds × 1,500 samples · round spread on base: 6 cycles (loop), 2 (decode)

BUILD	PID LOOP	VS BASE	MOTOR UPDATE	DECODE
base	8,033	—	5,794	1,713
E (ITCM swap)	**7,376**	**−8.2%**	**5,165 (−10.9%)**	**1,563 (−8.7%)**
D (decoder)	7,813	−2.7%	5,589 (−3.5%)	1,532 (−10.5%)
A2 (scheduler)	7,940	−1.2%	5,698	1,720
C (mixer)	7,958	−0.9%	5,702	1,712
B (hold checks)	8,034	0.0%	5,791	1,717
Sol 3 (separate run, its own base 8,032)	7,863	−2.1%	5,683	1,707

E and D carry most of the gain. The four single-patch savings add to about 1,045 cycles against 962 for all patches together, so they roughly add up. MEASURED

> **Read the small numbers with care.** The same firmware gives almost the same number every boot (spread of a few cycles), but any rebuild moves code around in flash and changes cache and wait-state behaviour. A2 only touches the scheduler, yet gyro update moved −8%; C is a no-op on F722, yet the loop moved −0.9%. So a change of about 1–2% in one stage can be a layout effect, not the patch. The round spread understates the real noise. HYPOTHESIS for the cause; the moves themselves are MEASURED.
>
> Sol 3's −2.1% is real for default settings (gyro filtering −2.3%, PID −4.1%: the default config has these notches off), but under the worst-case rule it is a best-case gain.

Rule from the Betaflight devs: optimize for the worst case

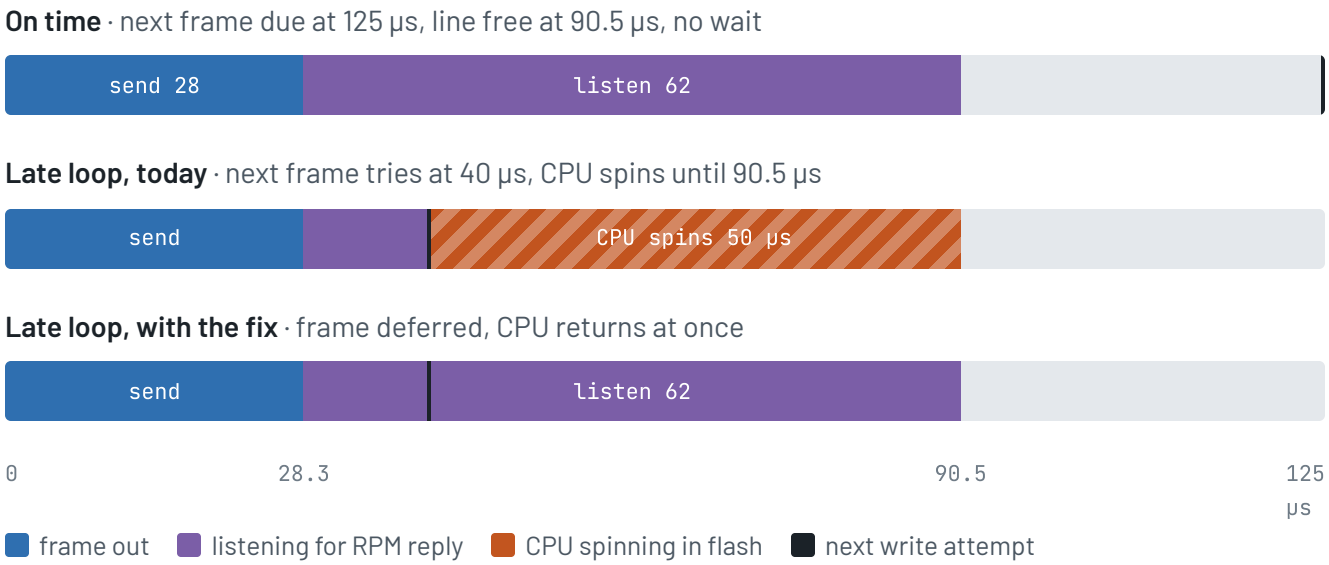
> QuickFlash on patch C: skipping math when a feature is off makes the loop faster only for pilots who do not use it. It hides the true worst-case loop and adds a small cost for pilots who do. Optimize the worst case, not the best case.
>
> What we do with it: drop C; quote Sol 3 as best case only (it changes how an existing skip works, it does not add one); keep the patches that cut work that always runs (D, A2); bench a worst-case setup (thrust linear on, all RPM harmonics, dynamic notch) and log the maximum loop time, not only the average.

The don't-spin fix

This is the change that matters most for 8 kHz. All three model lanes (Opus, GPT 5.6 Sol, Fable) worked the timing problem independently and ranked the same fix first.

The problem

After each motor frame, the DShot pins are busy for 90.5 µs: 28.3 µs sending the frame (51 states at 1.8 MHz), then 62.2 µs listening for the ESC's RPM reply (140 samples at 2.25 MHz). The listen phase cannot be cut short: aborting it drove the line against a transmitting ESC and corrupted the barometer (#15533). READ `dshot_bitbang.c:408-425`



At a steady 8 kHz the line is free 34.5 µs before the next frame, so nothing waits. But if a loop runs late, the next write lands while the line is still busy. Today `bbTelemetryWait` spins in flash until the reply ends: up to 62 µs, with a 77 µs backstop. Add about 61 µs of loop work and a bad loop takes about 123 µs, the whole budget. The next gyro sample is missed, the next loop starts late, and the delay can feed on itself. That is what #15779 guards against by dropping to DShot300 and 4 kHz. HYPOTHESIS for the feedback effect; the spin itself is READ `dshot_bitbang.c:513-556`

The fix

- Before touching the buffers, check whether the previous frame still owns the line (sending, or listening, within a 105 µs guard from the send).
- If it does, mark this frame deferred and return at once. Leave the buffers, the telemetry request and the DShot command queue untouched for the next loop.
- The ESC keeps its previous throttle for one more 125 µs loop. ESC signal timeouts are in milliseconds, so this is invisible to the motor.
- The reply is never aborted, so #15533 stays fixed. A stream stuck past the guard is reset the same way the old backstop did.
- It also closes a second bug: two writes less than 28 µs apart used to rewrite the output buffer while DMA was still sending it.

Worst-case loop: about 123 µs today, about 56 µs with the fix and the patches. The loop never drops a gyro sample because of DShot. HYPOTHESIS

Three independent versions

Opus

New `bbTelemetryBusy()`, busy also during the send phase. ITCM +80 B. Unit tests: motor_output 1, scheduler 13, mixer_wing 2 pass.

GPT 5.6 Sol

Age from the last send, 105 µs guard, marks stalled captures for reset. ITCM 15,744 B. motor_output test passes.

Fable

`bbLineBusy()` also checks the DMA stream; adds a "deferred frame" debug counter. ITCM +176 B. motor_output and scheduler tests pass.

All three build for SPEEDYBEEF7V3 and GEPRCF722_BT_HD_V3 and change only `src/platform/STM32/dshot_bitbang.c`. No unit test covers the bitbang driver itself. All three booted on the SpeedyBee in a smoke test; none has been tested at 8 kHz or flown. `MEASURED` builds

What it does not do

- It does not reduce CPU load. It turns a 0–77 µs spin into one skipped motor update.
- If loops stay closer than 90.5 µs for a long stretch, the motor update rate halves without warning. The probe build counts deferred frames so we can see it.
- Other platforms (APM32, AT32, GD32, X32) keep the spin.

How we prove it

- The SpeedyBee bench board cannot show it: its BMI270 runs at 3.2 kHz, so a loop is 312 µs and the 90.5 µs busy window never meets the next write.
- Proof needs the GEPRC F722 (ICM-42688P, real 8 kHz) with ESCs connected. Flash the probe build with and without the fix and compare spin time, deferred frames, missed gyro samples and telemetry error rate.
- Then merge the best parts of the three versions into one patch.

Open questions

- **Shared DMA2.** On F722 the gyro SPI DMA and the bitbang DMA share the DMA2 controller. On F4, Betaflight keeps SPI DMA off DMA2 with bitbang because of an errata item. The F7 code has no such guard. We have not yet read the F72x errata sheet (ES0360); ST's site refused the download. If the item applies, the corrupted side is likely the GPIO output, not the gyro. `HYPOTHESIS`
- **Decode race.** Decoding the last reply runs while the next capture starts filling the same buffer 28 µs later. If decode takes longer than that, RPM telemetry (not gyro data) is corrupted. Patch D makes decode faster, which shrinks the risk. `HYPOTHESIS`

- **Worst-case bench.** Thrust linear on, all RPM harmonics, dynamic notch on, with maximum loop time logged.
- **8 kHz hardware.** All of the above needs the GEPRC board and ESCs.

How each number was measured

NUMBER	METHOD
Stage cycles (PID loop, motor update, decode, filters)	A TEST ONLY build reads the CPU cycle counter before and after each stage, averages 1,024 calls, and puts the result in `debug[]`. A script reads it over MSP with `debug_mode = NONE`. Each build is flashed by DFU with a full erase. Rounds are interleaved (base, patched, base, …) so drift hits both equally. 216 MHz converts cycles to µs.
Scheduler pass cost (A2)	Same method around the task-priority loop in `scheduler()`, plus a count of scheduler passes per 100 ms. 3 rounds × 1,500 samples.
Same output as master	Disassembly of the whole firmware with the revision string pinned, compared line by line. 0 differing lines means the same machine code. A2's priority result was also compared with master on 1.3 billion host cases.
Flight behaviour	SimITL runs the real firmware as SITL against a simulated quad and a scripted flight. The traces, CLI output and blackbox must be byte-identical to master. Deliberate small breaks must change them, which proves the test can see a change.
Decoder correctness (D)	The new and old decoder decode the same input: a 2,209-frame corpus and 20 million random buffers, 0 mismatches.
The spin and the 62 µs window	Read from the driver: 51 output states at 1.8 MHz and 140 input samples at 2.25 MHz. Not yet measured at 8 kHz. The `DSHOT_BB_PROBE` build measures it on a board.

Build switches: what each one buys

Branch `f722-bidir-switchable` puts every patch and all three spin fixes behind build options, so you can test them one at a time or together. With no switches it builds master. Turn things on with `make CONFIG=<board> EXTRA_FLAGS="-D<switch> …"`.

Built and checked cyclistmass/betaflight · f722-bidir-switchable

Savings are PID-loop cycles on the SpeedyBee F7 V3 (base 8,033 cycles = 37.2 µs) unless the row says otherwise. Source column: where the number came from.

SWITCH	PATCH	ESTIMATED SAVING	WHERE IT CAME FROM
PERF_ALL	**Loop fixes + spin fix**	**Average: loop −12%, ~4.7 µs per loop. Worst case: ~123 → ~56 µs**	**Loop: board, 5 rounds MEASURED. Worst case: code reading HYPOTHESIS**
PERF_LOOP	**A2 + B + D + E**	**Loop −12.0% (8,033 → 7,071). CPU at 8 kHz ~49% → ~45%**	**Board, 5 interleaved rounds, all patches incl. C (C adds ~0 on F722) MEASURED; 8 kHz share projected HYPOTHESIS**
PERF_NOSPIN	**Spin fix only (Fable version unless one is named)**	**Removes up to 62 µs of spin (77 µs backstop) from a late loop**	**Driver timing from source constants READ; effect on a real 8 kHz loop not yet measured**
PERF_SCHED_TASK_AGE	A2 scheduler	−21 cycles per scheduler pass (−3.3%), +9.3% passes. Loop itself −1.2%, mostly code layout	Board scheduler probe, 3 rounds MEASURED; same output as master on 1.3 billion cases and in the simulator
PERF_HOLD_TASK_SIGNAL	B hold tasks	0 on a quad without GPS/baro hold. Saves polling only when alt/pos hold runs	Board probe shows 0 here MEASURED; with hold active not measured
PERF_MIXER	C mixer (not in any preset)	~0 on F722. Best case only: helps only when thrust linear is 0	Map/disassembly: compiler already hoists it MEASURED; simulator −45 x86 instructions per loop
PERF_BB_DECODE	D decoder	Decode −10.5% (1,713 → 1,532), loop −2.7%	Board, 3 rounds MEASURED; simulator decoder −34% instructions, 20 M fuzz buffers 0 mismatches

`PERF_BB_ITCM`	E ITCM swap	Loop −8.2% (8,033 → 7,376), motor update −10.9%. The largest single gain	Board, 3 rounds `MEASURED`; no simulator (no ITCM in SITL)
`DSHOT_BB_NOSPIN_OPUS` (=1)	Opus spin fix	Same target as above. ITCM +80 B	Builds, 3 unit tests pass, boots on SpeedyBee `MEASURED`; not tested at 8 kHz
`DSHOT_BB_NOSPIN_SOL` (=2)	GPT 5.6 Sol spin fix	Same target. ITCM +32 B	Builds, 1 unit test passes, boots on SpeedyBee `MEASURED`; not tested at 8 kHz
`DSHOT_BB_NOSPIN_FABLE` (=3)	Fable spin fix, counts deferred frames in debug[4]	Same target. ITCM +176 B	Builds, 2 unit tests pass, boots on SpeedyBee `MEASURED`; not tested at 8 kHz

How the switches were checked

CHECK	RESULT
Builds: every preset, every single switch, the presets with each patch taken out, both boards (SpeedyBee F7 V3, GEPRC F722)	All build. Two spin fixes at once, or a bad value, stop the build with a clear error
Same machine code (revision pinned): switches off vs master; `PERF_LOOP`+`PERF_MIXER` vs the patch branch; `PERF_ALL`+`PERF_MIXER`+Opus vs the Opus lane's tree	**Identical, 0 lines**. Controls: A2 on vs master and patches vs switches off both differ
Unit tests, with switches off and as `.perf` variants with them on (scheduler, althold, poshold, autopilot_wing) plus motor, mixer and PID tests	All pass
Simulator, calm scripted flight (arm, stick work, 20–60% throttle, RC loss with auto-land, re-arm): A2, C, D, A2+C+D vs switches off, thrust linear 0 and 25	**21/21 identical** (trace, CLI, blackbox). D on the bitbang sim with 0% and 20% damaged replies: 4/4 identical
Simulator, rough flight (seeded gusts, 100% punch-outs, full-stick flips, airmode chops, motor imbalance, battery sag)	**8/8 identical** over 304,038 scheduler passes each

Every spin fix with the timing probe on, `-Werror`: F405, F411, F722, G474, H743, AT32F435, APM32F405, GD32H757, X32M7, SITL	All build
Betaflight CI on the fork (clang, every SDK and target, `checks`, `test-all`), commit 900f67102	**54/54 jobs pass**
Board smoke test, SpeedyBee F7 V3: all five test builds flashed, 300 samples of `DSHOT_TELEMETRY_COUNTS` each	All boot and keep DShot600, bidir and denom 1. 0 reception timeouts. The Fable fix deferred 3 frames
Red controls that must differ: old patch A, a deliberate +1 in the mixer, a bit-flip in the decoder, a different gust seed	All differ, so the comparisons can see a change

The rough flight did not reach any new branches in the patched code (branch coverage the same as the calm flight). The branches still not reached depend on configuration (3D mode, missing telemetry), not on how hard the quad is flown. The simulator cannot show speed, ITCM placement (E) or DMA timing (the spin fixes); those need a board. `MEASURED`

Take one patch out of a preset with `-DNO_PERF_<name>`, or keep the spin with `-DDSHOT_BB_NOSPIN=0`. Naming two spin fixes at once is a build error. To run 8 kHz with bidir on F722, also add the TEST ONLY `-DTEST_UNGATE_PID_DENOM_CHECK`, which lifts the #15779 downgrade; never release it. The per-patch savings do not add exactly (about 1,045 cycles alone against 962 together) because each rebuild moves code in flash.

Where the work lives

- **Hand-off branch with build switches:** cyclistmass/betaflight · f722-bidir-switchable (all off = master; presets and switches in `src/main/build/perf_switches.h`)
- **How to test it:** F722-PERF-TESTING.md · this page as markdown: F722-PERF-FINDINGS.md
- **Test firmware** (GEPRCF722_BT_HD_V3, SPEEDYBEEF7V3, five builds each): f722-test-900f67102
- Patches, one commit each: cyclistmass/betaflight · f722-bidir-perf
- Bench scripts, raw results and findings: cyclistmass/betaflight · f722-bidir-bench (test-only guard; small sample set)
- Rule being lifted: betaflight #15779 · capture-abort history: #15533

Compiled 14 Oct 2026, updated 15 Oct 2026, from board runs, simulator profiles, CI and the three DShot timing lanes. — Cyclistmass